studocu

SC2006 Final Exam Notes

Software Engineering (Nanyang Technological University)



Scan to open on Studocu

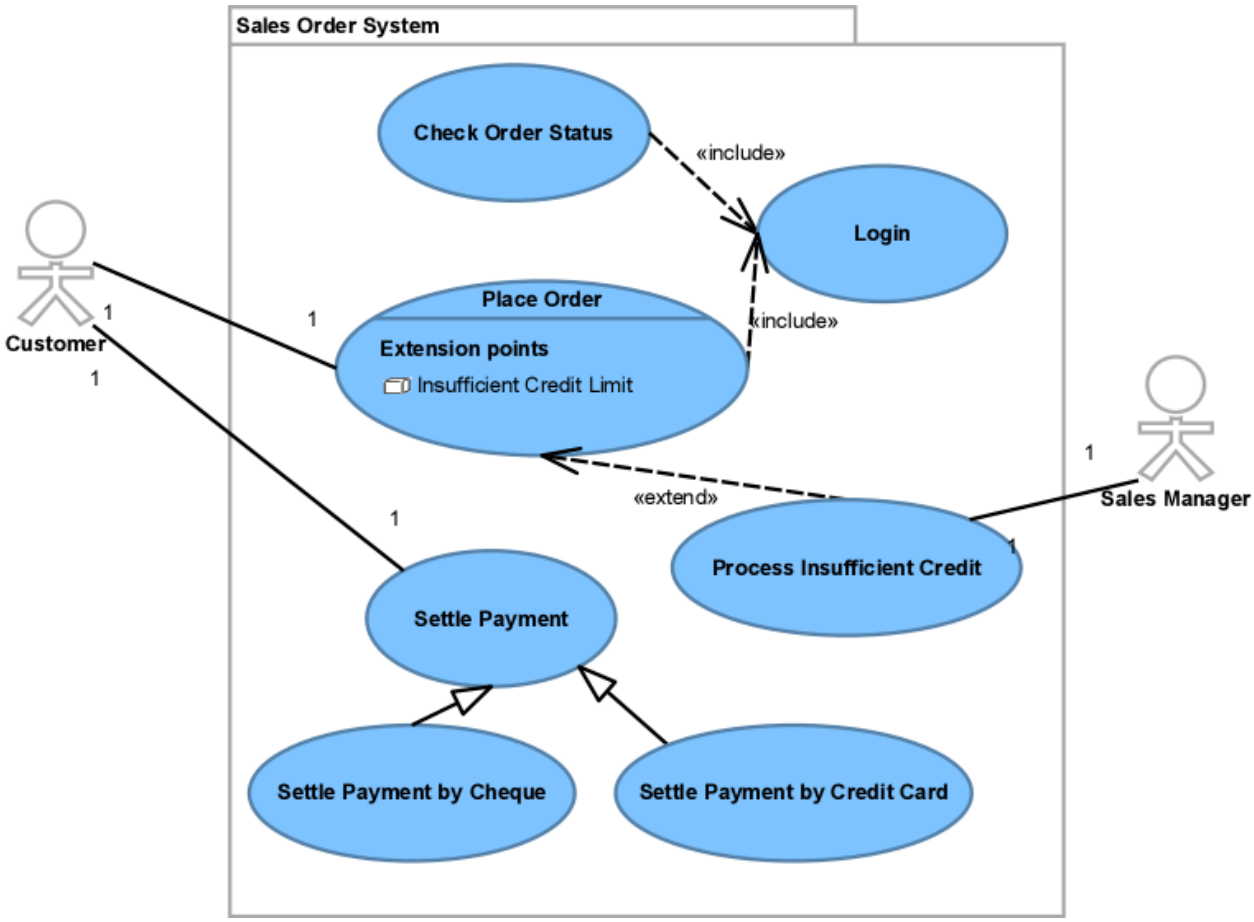
Studocu is not sponsored or endorsed by any college or university
Downloaded by The Mythiology (themythiology@gmail.com)

ALL DIAGRAMS SC2006

---

1. Use Case Diagram

- Actors (stick figures)
- Use cases (ovals)
- System boundary (rectangle)
- Arrows:
    - Association (solid line)
    - <<include>>, <<extend>>
      Actor generalization (triangle arrow)



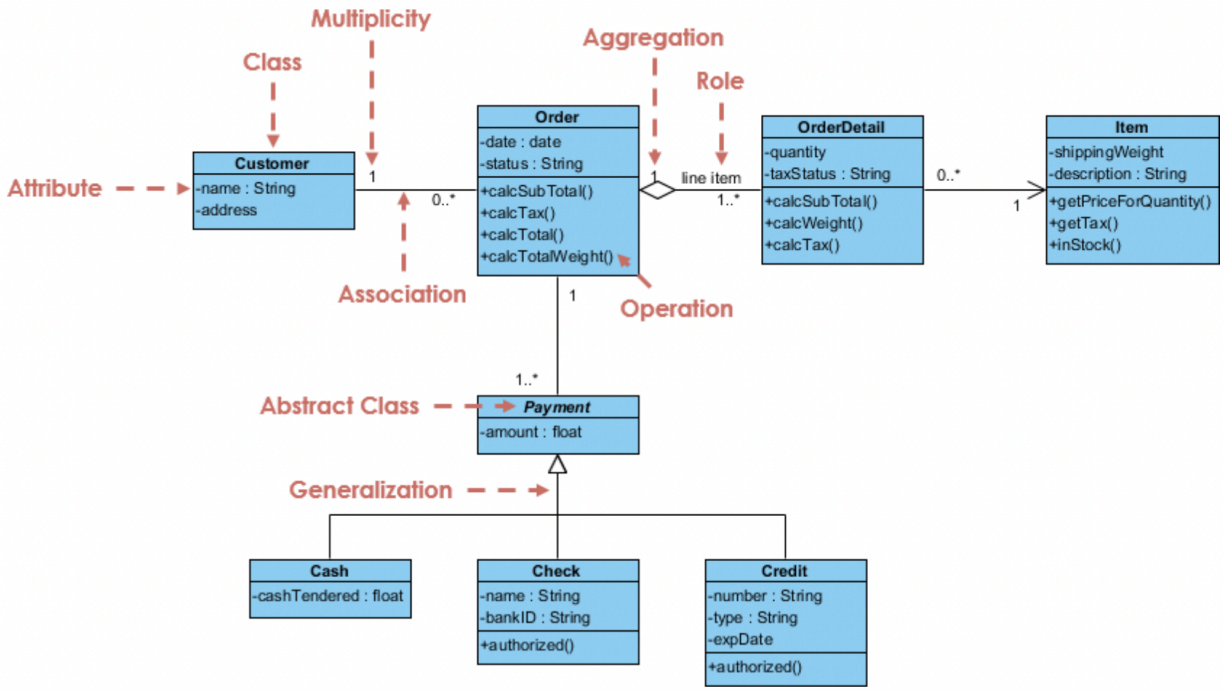
This document is available on studocu

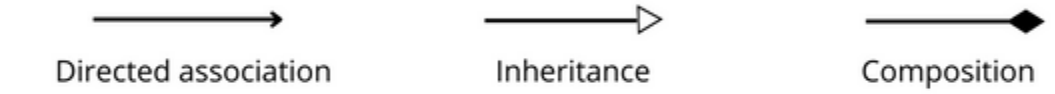
Downloaded by The Mythiology (themythiology@gmail.com)

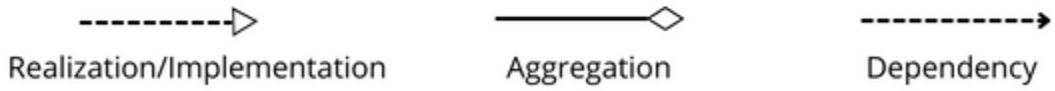
2. UML Class Diagram

- Classes: Name, attributes, methods
- All arrow types:
    - Association
        - Syntax: Solid line between two classes
        - Meaning: Structural relationship; one class "has-a" reference to another
        - Direction: Optional arrow for unidirectional association
        - Used when: One object stores or uses another
        - Example: A Student is associated with a Course
    - Inheritance (triangle)
        - Syntax: Solid line with hollow triangle pointing to the parent class
        - Meaning: "Is-a" relationship; child inherits from parent
        - Used when: Modeling polymorphism or abstract class hierarchies
        - Example: Manager inherits from Employee
    - Aggregation (white diamond)
        - Syntax: Solid line with a white diamond at the "whole" side
        - Meaning: "Has-a" relationship; part can exist independently
        - Used when: Part object can exist without the whole
        - Example: A Library aggregates Books, but books can exist elsewhere
    - Composition (black diamond)
        - Syntax: Solid line with a black diamond at the "whole" side
        - Meaning: Strong "has-a"; part cannot exist without the whole
        - Used when: Part's lifetime is tied to the whole
        - Example: An Order is composed of OrderItems that die with the Order
    - Dependency (dashed arrow)
        - Syntax: Dashed line with arrow pointing to the used class
        - Meaning: Temporary "uses" relationship; no stored reference
        - Used when: A class uses another only inside a method
        - Example: Controller depends on EmailService during execution

Downloaded by The Mythiology (themythiology@gmail.com)



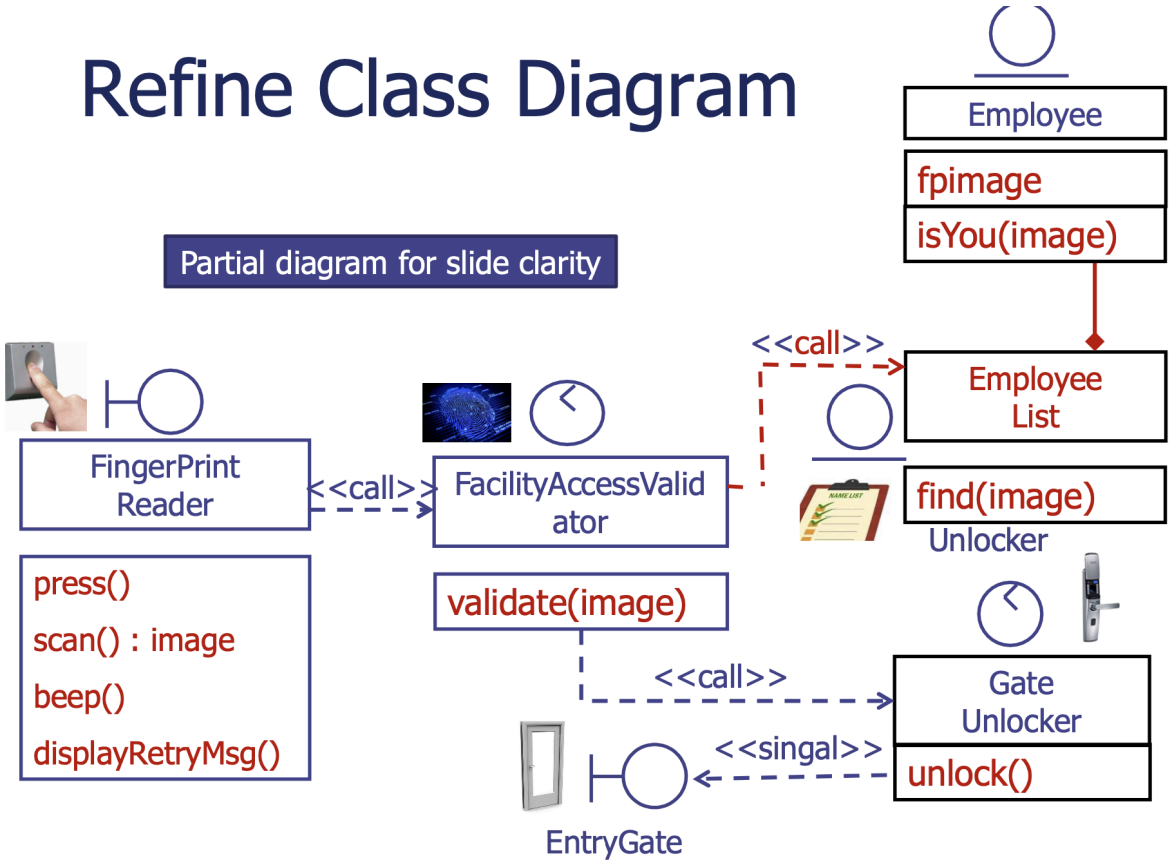




This document is available on studocu
Downloaded by The Mythiology (themythiology@gmail.com)

3. Class Stereotype Diagram (BCE Model)

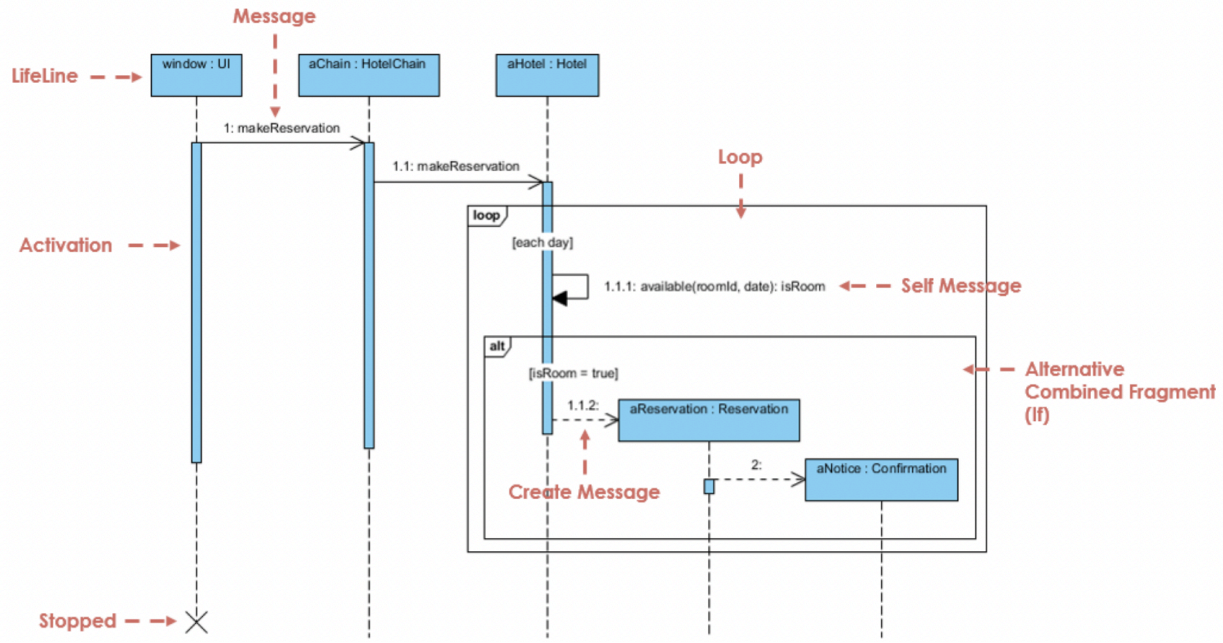
- No attributes or methods unless asked
- Use arrows labeled with roles: <<calls>>, <<updates>>, <<stores>>



Downloaded by The Mythiology (themythiology@gmail.com)

4. Sequence Diagram

- Lifelines (vertical dashed lines)
- Activation bars (thin rectangles)
- Message arrows (→)
- Return arrows (optional)
- <<create>>, <<destroy>> if needed

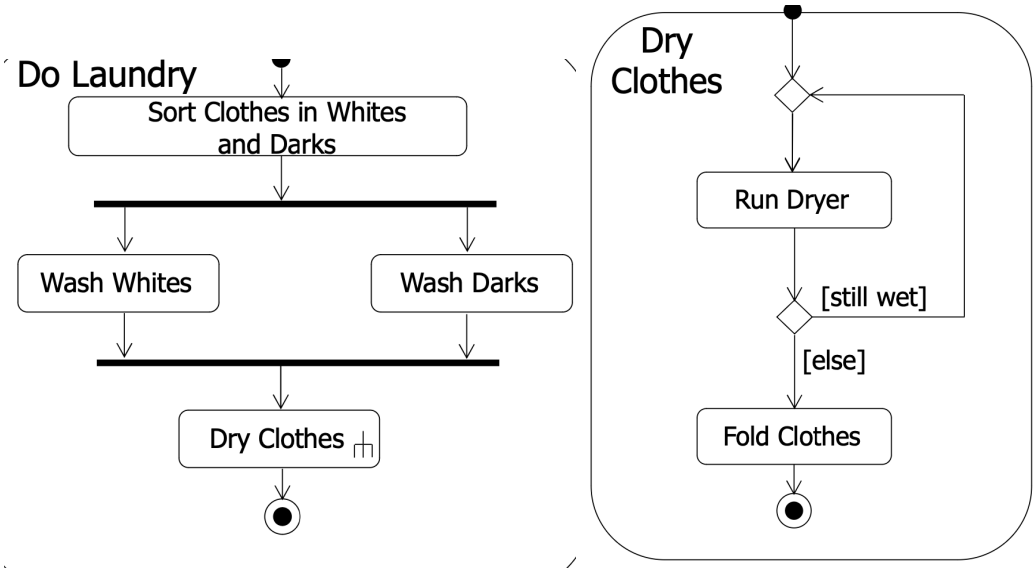


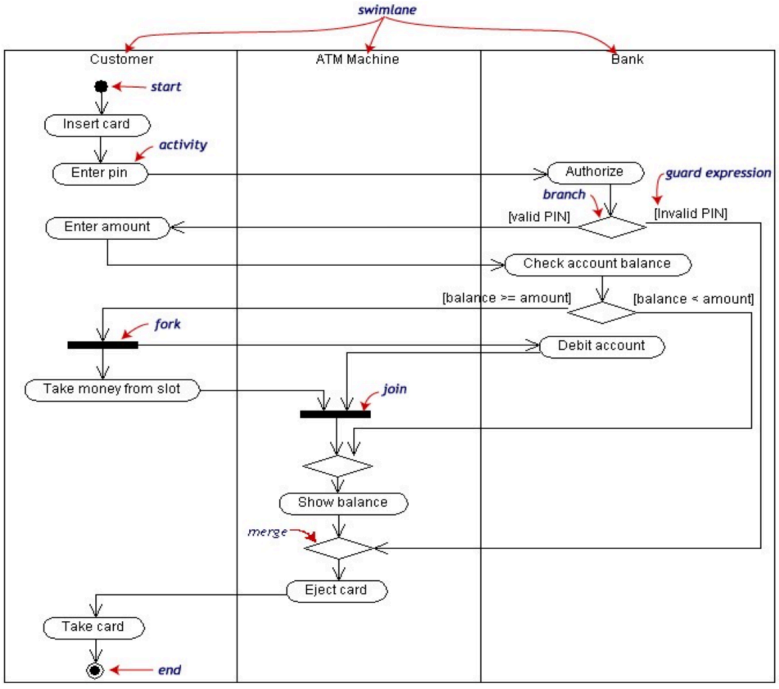
This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

5. Activity Diagram

- Start (●), End (◎)
- Actions (rounded rectangles)
- Decisions/Merges (diamonds)
  Arrows for flow
- Loops/back edges if needed

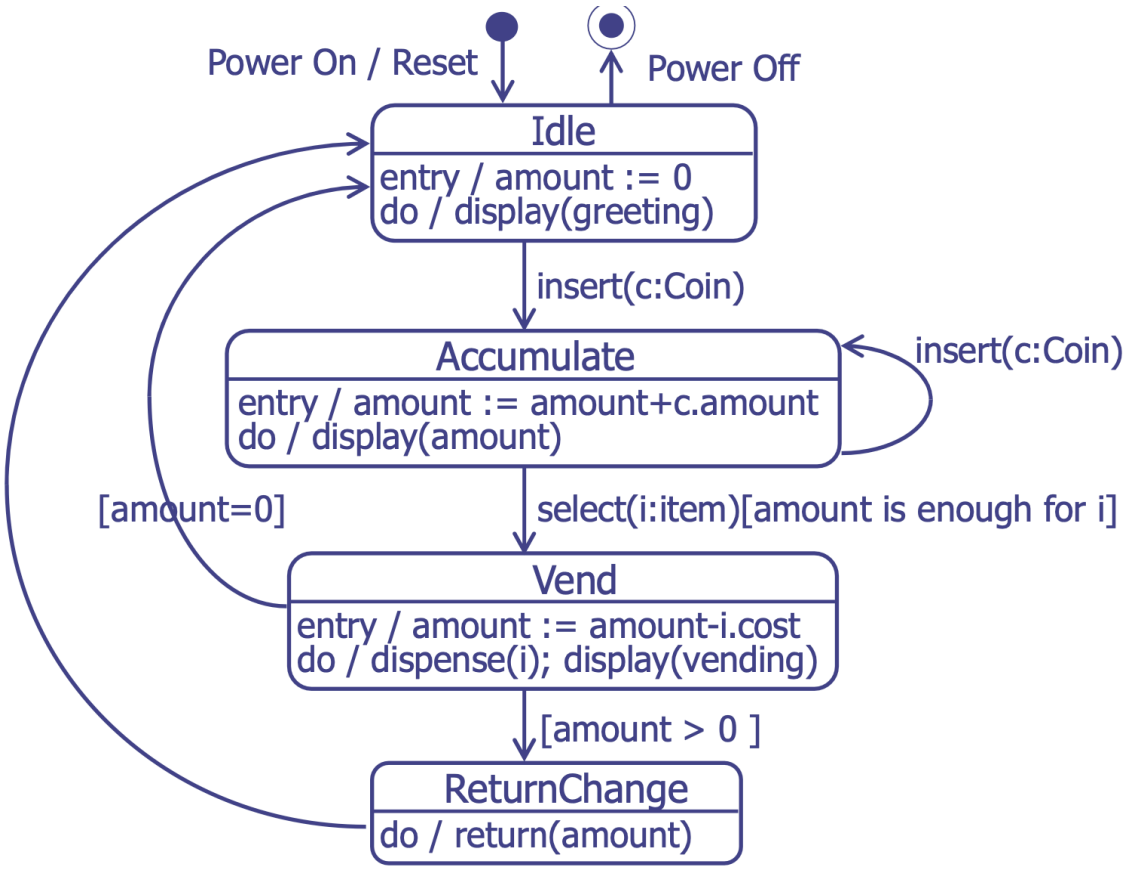




Downloaded by The Mythiology (themythiology@gmail.com)

6. Dialog Map / State Machine Diagram

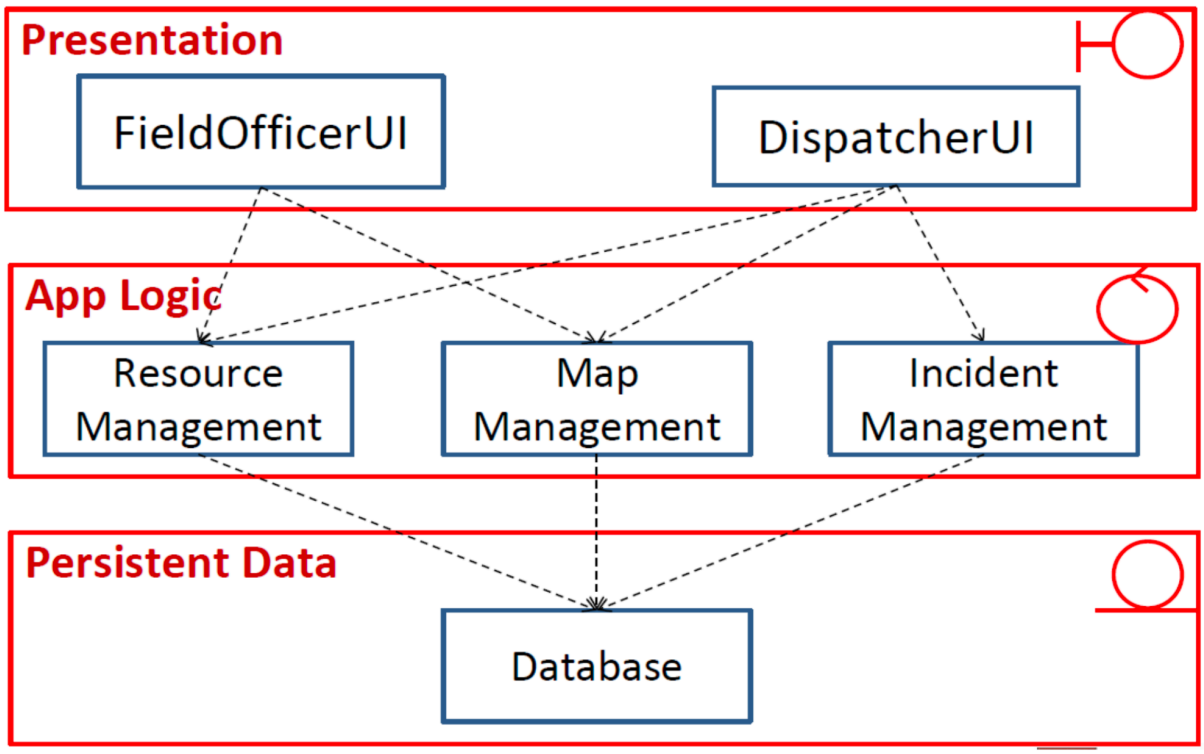
- States (rounded rectangles)
- Transitions (arrows)
- Labels like entry /, do /, exit /
- Start (●), End (◎)



This document is available on studocu
Downloaded by The Mythiology (themythiology@gmail.com)

7. Architecture Diagram

- Boxes representing components or layers (e.g., MVC, 3-Tier)
- Arrows for communication/dependencies

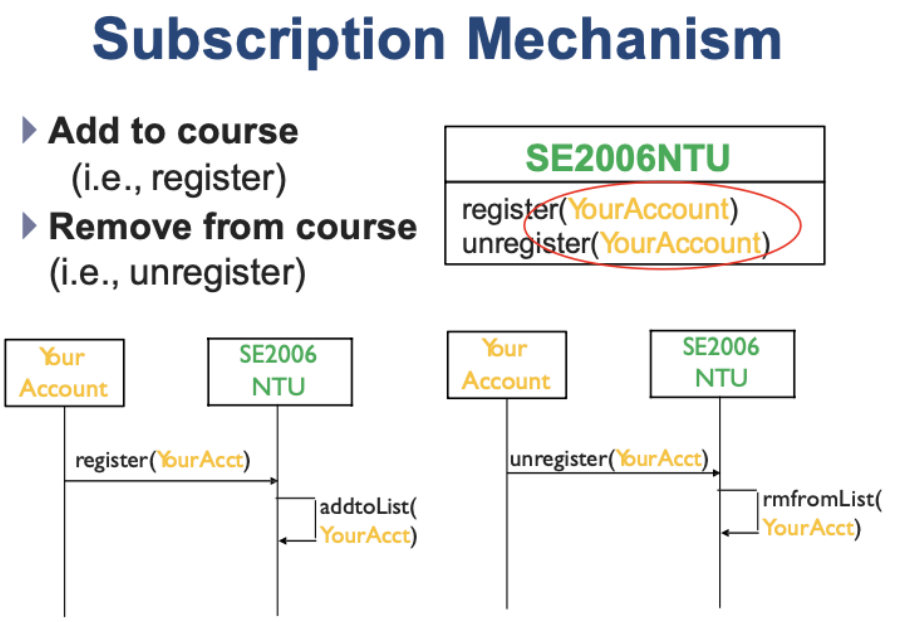


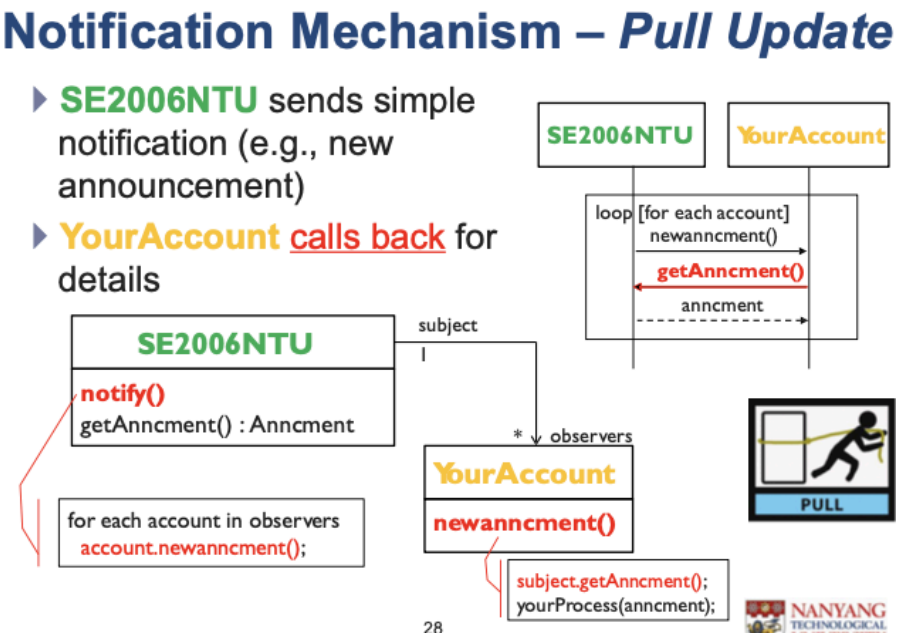
Downloaded by The Mythiology (themythiology@gmail.com)

8. Design Pattern Diagrams

a. Observer Pattern

- Subject + Observer interfaces
- ConcreteSubject and ConcreteObserver
- Arrows:
    - Aggregation (Subject has observers)
    - Method call: update(), notify()

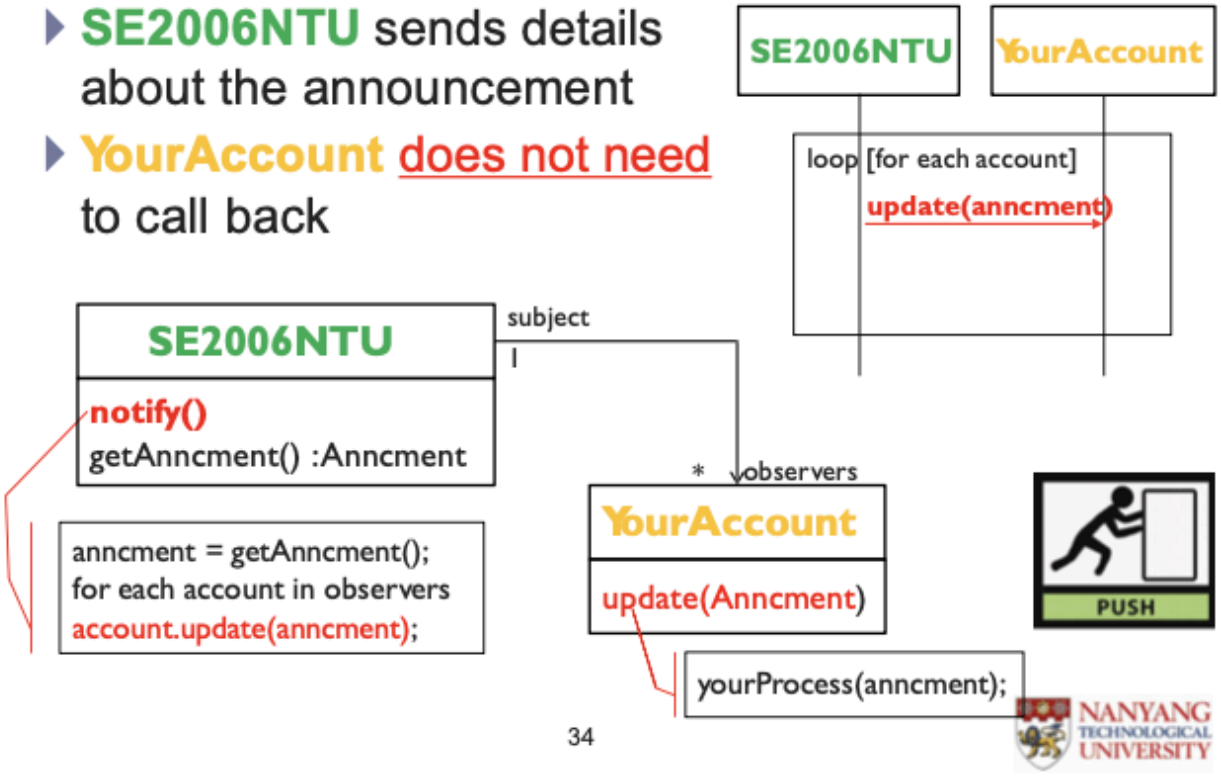




This document is available on studocu
Downloaded by The Mythiology (themythiology@gmail.com)

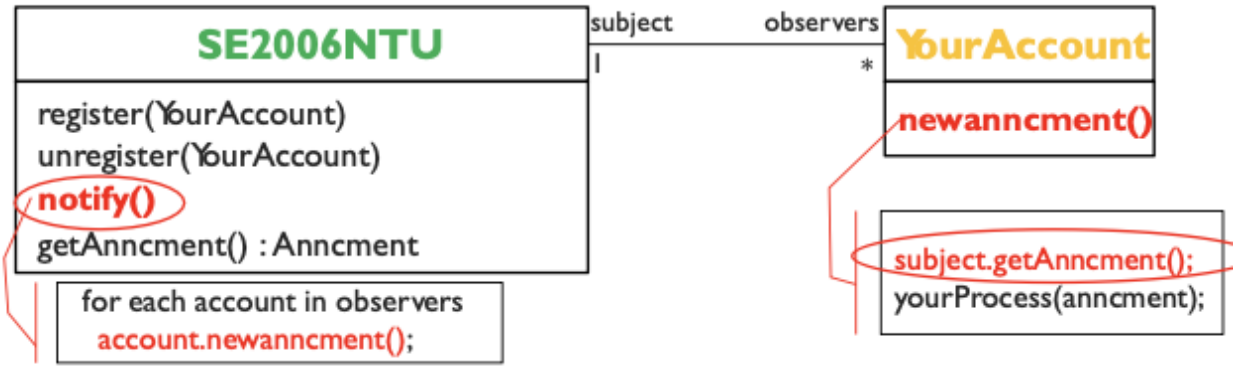
Notification Mechanism – *Push Update*

- **SE2006NTU** sends details about the announcement
- **YourAccount** does not need to call back



NANYANG TECHNOLOGICAL UNIVERSITY

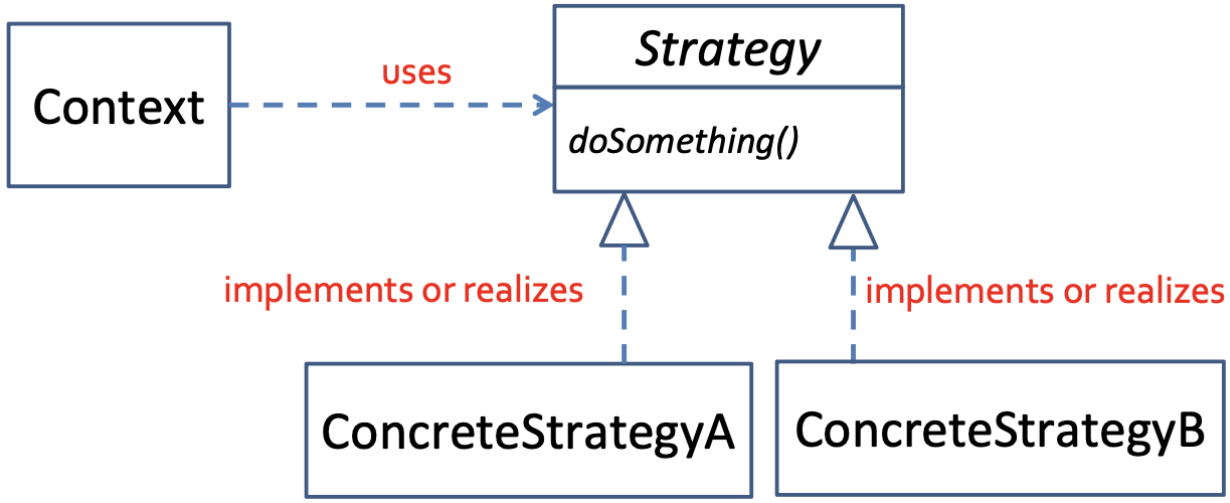
Pull it Together in a Class Diagram



Downloaded by The Mythiology (themythiology@gmail.com)

b. Strategy Pattern

- Strategy interface
- Concrete strategies
- Context class using the strategy
- Arrows:
    - Association from context to strategy



c. Factory / Facade (on other paper)

- **Factory**: Briefly mentioned, not exam-priority
- **Facade**: One slide mention – not required to draw unless asked explicitly

---

9. Control Flow Graph (CFG)

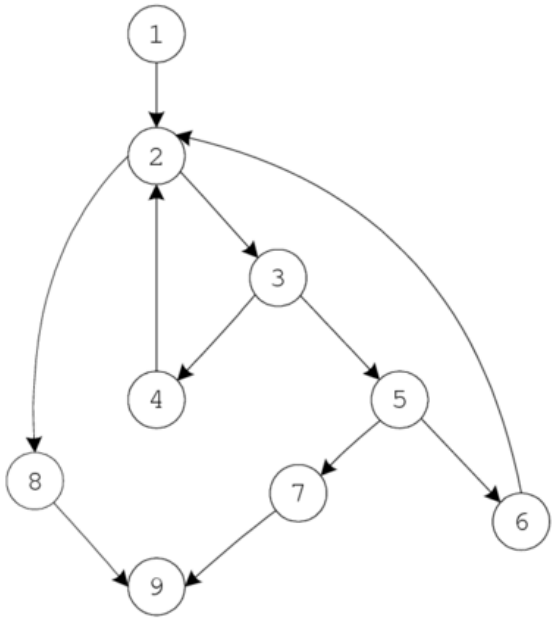
- Nodes: statements or conditions
  Arrows: control flow
- Must know how to:
    - Count nodes and edges
    - Calculate Cyclomatic Complexity
      List basis paths

This document is available on studocu
Downloaded by The Mythiology (themythiology@gmail.com)

Source Program:

```
int binsearch(int x, int v[], int n)
{
      | int low, high, mid;
   1  | low = 0;
      | high = n - 1;
        while (low <= high) | 2
        {
            3 | mid = (low + high)/2;
              | if (x < v[mid])
                      high = mid - 1;  | 4
            5 | else if (x > v[mid])
                      low = mid + 1;   | 6
            7 | else return mid;
        }
        return -1; | 8
} | 9
```

CFG:



Downloaded by The Mythiology (themythiology@gmail.com)

SC2006 SYSTEM ARCHITECTURE PATTERNS

---

1. Three-Tier Architecture

(**Presentation → Application → Data**)

Description:

Separates the system into three layers:

- **Presentation Layer**: UI (web, mobile)
- **Application Layer**: Business logic, controller
  Data Layer: Database, data access

Use Case:

- Web apps, booking systems, online forms

Pros:

- Separation of concerns
- Maintainable and modular
- Each tier can scale independently
- Easier to test and debug

Cons:

- Not very dynamic (UI can't auto-update)
- Slower due to communication overhead
- Hard to bypass intermediate layers
- Can lead to over-layering in simple apps

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

2. Model-View-Controller (MVC)

Description:

Separates responsibilities for dynamic, interactive applications:

- **Model**: Manages data (e.g., list of bookings)
- **View**: UI (e.g., table showing bookings)
- **Controller**: Handles user actions (e.g., button click)

Use Case:

- GUI apps (e.g., Eclipse), web frameworks (Django, Rails)

Pros:

- High modularity and separation
- Supports parallel development (frontend/backend)
- Supports UI auto-refresh with Observer pattern
- Good for testability and UI flexibility

Cons:

- Tight interaction between components
- Increased complexity
- Harder to debug with many views/controllers

---

3. Layered Architecture (similar to Three-Tier, but more general)

Description:

Organizes the system into **horizontal layers**, e.g.:

- UI
- Business Logic
- Data Access
- Framework / Utilities

Use Case:

Downloaded by The Mythiology (themythiology@gmail.com)

- Enterprise systems, government software

Pros:

- Clean separation
- Replace layers independently
- Teams can work on layers in parallel

Cons:

- Can lead to tight coupling between adjacent layers
- Performance overhead (requests pass through many layers)
- Can't easily skip layers (e.g., UI can't access DB directly)

---

4. Client-Server Architecture

Description:

- **Client** requests services (e.g., browser)
- **Server** provides services (e.g., web server, DB server)

Use Case:

- Web apps, mobile apps using cloud services

Pros:

- Centralized control and updates
- Easy to deploy and scale server
  Simple to implement

Cons:

- Single point of failure (server)
- Dependent on network quality
- Clients can't work offline (usually)

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

5. Pipe-and-Filter Architecture

Description:

System is divided into **filters (processing units)** connected by **pipes (data channels)**. Each filter processes data and passes it on.

Use Case:

- Compilers, data processing pipelines, signal processing

Pros:

- Highly reusable components
- Easy to understand and modify
- Supports streaming data

Cons:

- Not good for interactive systems
- Hard to handle shared state
  Requires uniform data formats

---

6. Service-Oriented Architecture (SOA)(a precursor to Microservices)

Description:

- Software is organized into **loosely-coupled services**
- Services expose interfaces over a network (e.g., REST API)

Use Case:

- Large enterprise systems, cross-platform integration

Pros:

- Language-agnostic (can use Java, Python, etc.)
  Encourages reuse and modularity

Downloaded by The Mythiology (themythiology@gmail.com)

- Good for distributed systems

Cons:

- High communication overhead
- Complex security and service management
- Not performant for small-scale systems

---

7. Observer-Based Component Architecture

Description:

- Uses the **Observer Pattern** across UI components
- Components notify observers when their state changes

Use Case:

- Interactive systems, dashboards, notification-driven apps (e.g., NTULearn notifications)

Pros:

- Decoupled components
- Supports real-time updates
- Ideal for reactive UIs

Cons:

- Hard to trace control flow (events are indirect)
- Risk of memory leaks (if observers not deregistered)
- Complex when many observers are registered

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

---

8. Plugin-Based / Dynamic Component Loading

Description:

System loads components **at runtime** using dynamic class loading. Often paired with Factory or Strategy Pattern.

Use Case:

- IDEs (e.g., Eclipse), browser extensions

Pros:

- Extremely extensible
- Supports runtime behavior changes
- Isolates code into modules/plugins

Cons:

- Debugging is difficult
- Plugin compatibility and dependency issues
- Requires advanced architecture management

Summary Table – Architecture Types

Architecture	When to Use	Key Benefit	Key Drawback
Three-Tier	Web systems with clear UI-logic-data split	Modular, scalable	Slow communication between tiers
MVC	Interactive UIs, GUIs	UI flexibility, testability	Controller complexity
Layered	Enterprise-grade systems	Separation of concern	Performance overhead
Client-Server	Web/mobile apps	Simple, centralized control	Single point of failure

Downloaded by The Mythiology (themythiology@gmail.com)

Pipe-and-Filter	Data pipelines, compilers	Reusable, linear processing	Not interactive
SOA	Cross-system, large-scale systems	Language-independent, decoupled	Complexity, overhead
Observer-Based UI	Dashboards, reactive UIs	Decoupled, real-time updates	Hard to trace flow
Plugin-Based	IDEs, modular systems	Runtime flexibility	Harder to manage/debug

What's the Difference Between a Design Pattern and a System Architecture?

Category	**Design Pattern**	**System Architecture**
Purpose	Solves **specific code-level problems**	Defines the **high-level structure** of a system
Scope	Class/module level	Application/system-wide
Examples	Observer, Strategy, Factory, Facade	MVC, 3-Tier, Client-Server, Microservices
Reusability	Small, reusable logic component	Not reusable across systems as-is
Defined In	GoF Book (Gang of Four Patterns)	Software engineering frameworks
Diagrams?	Yes — mostly **UML class diagrams**	Yes — typically **box-layer diagrams**

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

SC2006 DESIGN PATTERNS

1. Observer Pattern

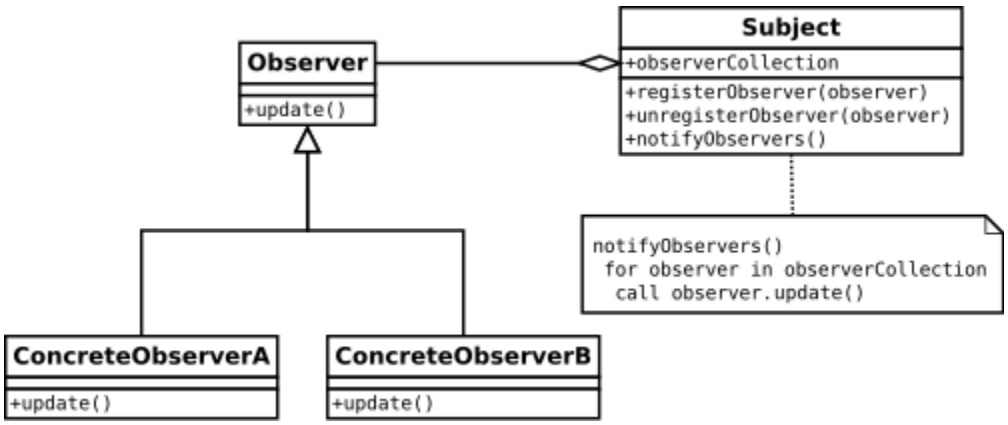
Purpose:

One-to-many dependency: when one object (the Subject) changes, **all registered Observers are notified** automatically.

Structure:

Component	Responsibility
Subject	Maintains a list of observers
Observer	Defines update() interface
ConcreteSubject	Implements subject; holds actual data
ConcreteObserver	Implements update behavior

Diagram:



Pros:

- Loose coupling between sender and receivers
- Dynamic runtime behavior (observers can be added/removed)
- Scalable for broadcasting events

Downloaded by The Mythiology (themythiology@gmail.com)

Cons:

- Complex for large observer chains
- Harder to debug/troubleshoot who got notified
- Potential memory leaks (if observers not unregistered)

---

2. Strategy Pattern

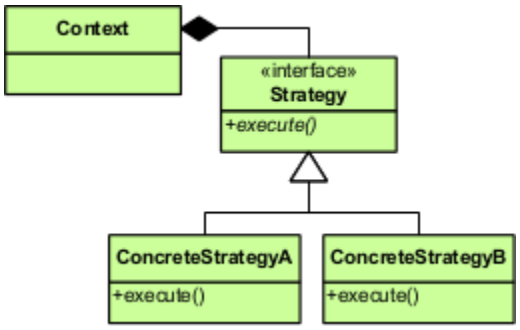
Purpose:

Enables you to **swap algorithms or behaviors at runtime** without modifying the class that uses them.

Structure:

Component	Responsibility
Strategy	Interface for all interchangeable algorithms
ConcreteStrategy	Each algorithm implementation
Context	Holds a strategy and delegates behavior to it

Diagram:



Pros:

- Avoids hardcoded logic or if-else chains
- Easy to extend with new behaviors
- Enables runtime flexibility

This document is available on studocu
Downloaded by The Mythiology (themythiology@gmail.com)

Cons:

- Introduces many small classes
- Client needs to understand strategy behavior
- Can increase complexity if overused

---

3. Factory Pattern

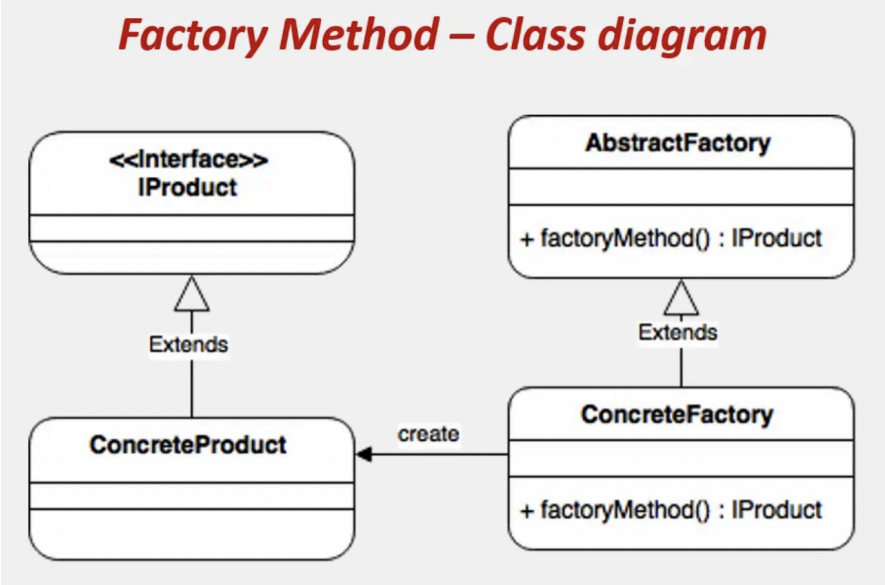
Purpose:

Encapsulates object creation logic, **allowing clients to request objects without knowing the exact class name**.

Structure:

Component	Responsibility
Factory	Defines method to create objects
Product Interface	Interface or superclass of products
Concrete Products	Implementations of the product

Diagram:



Downloaded by The Mythiology (themythiology@gmail.com)

Pros:

- Centralizes and simplifies object creation
- Promotes loose coupling
- Useful for plugin, extensible, or config-driven systems

Cons:

- Requires inheritance/interfaces
- Can hide dependencies or make code harder to trace
- Multiple factories can become confusing

---

4. Facade Pattern

Purpose:

Provides a **unified, simplified interface** to a complex subsystem (i.e., hides internal details behind one entry point).

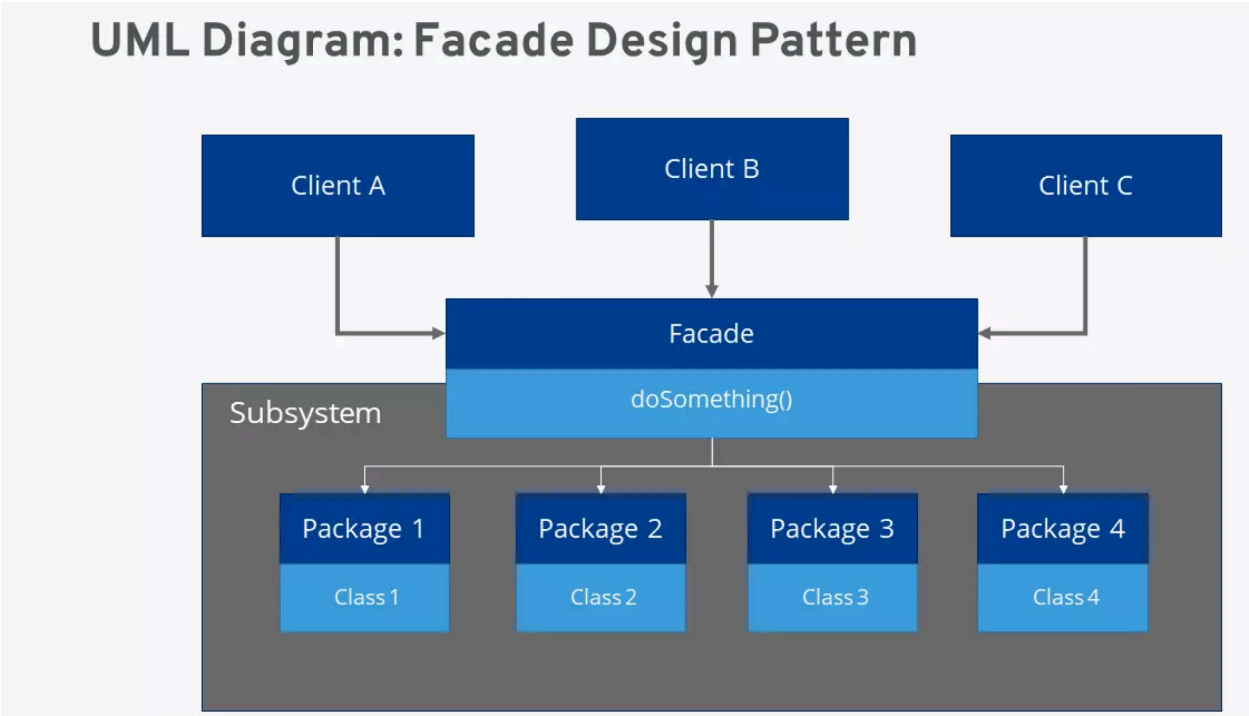
Structure:

Component	Responsibility
Facade	Provides high-level methods
Subsystems	Actual modules performing specific tasks

Diagram:

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)



Pros:

- Simplifies complex systems for clients
- Reduces coupling between subsystems and clients
- Makes code easier to use and understand

Cons:

- May hide useful features of subsystems
- Can become a "god class" if too bloated
- Doesn’t reduce system complexity — just hides it

Downloaded by The Mythiology (themythiology@gmail.com)

SC2006 SOFTWARE DEVELOPMENT PROCESSES

1. Waterfall Model

What It Is:

A **linear, sequential development model**. Each phase must be fully completed before the next begins.

Phases:

1. Requirements analysis
2. System design
3. Implementation
4. Testing
5. Deployment
6. Maintenance

Properties:

- Rigid, phase-based flow
- No overlapping or feedback loops
- Assumes stable requirements
- Emphasizes documentation

When to Use:

- Well-defined projects with clear, **unchanging requirements**
- Short projects where rework is expensive

Pros:

Strength	**Why It Matters**
Simple and easy to manage	Phases are clearly defined and structured
Well-documented	Each phase produces clear deliverables
Ideal for fixed-scope projects	Predictable timeline and budget

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

Cons:

Weakness	Impact
Poor adaptability to change	Any change means restarting the process
Late error detection	Testing comes late; bugs are found too late
No early feedback	Users only see the product at the end

2. V-Model (Validation and Verification)

What It Is:

A refinement of the Waterfall model that places **testing activities in parallel** with development phases.

Structure:

Properties:

- Every development phase has a **corresponding test phase**
- Emphasizes **early test planning**
- Verifies & validates system against requirements

When to Use:

- Safety-critical systems (e.g., medical, avionics)
- Where **testing is paramount**

Pros:

- Testing is integrated early
  Traceability of requirements to test cases
- Reduces risk of defect leakage

Downloaded by The Mythiology (themythiology@gmail.com)

Cons:

- Still rigid and linear
- High cost of change
- No working software until late stages

---

3. Incremental Model

What It Is:

Development and delivery occur in **small increments**, each adding functionality.

Structure:

[Increment 1: Login] → [Increment 2: Profile] → [Increment 3: Reports]

Each increment goes through:
Analysis → Design → Implementation → Testing

Properties:

- Prioritizes functionality delivery
- Modular, partial deployment
- Uses customer feedback to guide future increments

When to Use:

- Projects where delivery must begin early
- Evolving requirements
- Medium-scale systems

Pros:

- Delivers working software early
- Easy to isolate and fix problems

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

- Feedback-driven improvement

Cons:

- Architecture must handle future additions
- Planning across increments can be complex
- Integration effort may grow over time

---

4. Agile Development Model

What It Is:

A family of iterative and incremental models that emphasize **adaptability, collaboration, and working software**.

Agile is a **philosophy**, not a specific method. Scrum is the most widely used Agile **framework**.

Core Principles (from Agile Manifesto):

- Individuals and interactions > Processes and tools
- Working software > Comprehensive documentation
- Customer collaboration > Contract negotiation
- Responding to change > Following a plan

Properties:

- Iterative development (short cycles: 1–4 weeks)
- Frequent releases
- Continuous feedback
- Collaborative teams
- Requirements evolve over time

Pros:

- Flexible and adaptable

Downloaded by The Mythiology (themythiology@gmail.com)

- Continuous user feedback
- Early delivery of value
- Encourages collaboration and ownership

Cons:

- Hard to estimate time and cost
- Requires experienced, self-managing teams
- Can lead to scope creep
- May lack clear documentation if not managed

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

SC2006 SCRUM

What is Scrum?

Scrum is an **Agile framework** used for **iterative and incremental software development**, where the work is broken into fixed-length cycles called **Sprints** (typically 1–4 weeks).

Core Elements of Scrum

Roles

Role	Responsibility
Product Owner	Owns and prioritizes the Product Backlog. Acts as the voice of the customer.
Scrum Master	Ensures the team follows Scrum rules, removes impediments. Facilitator, not boss.
Development Team	Self-organizing, cross-functional team responsible for delivering usable features.

Events (Ceremonies)

Event	Timing	Purpose
Sprint	1–4 weeks	Time-boxed iteration for delivering a usable increment
Sprint Planning	Start of Sprint	Define the Sprint Goal; select stories from the Product Backlog
Daily Scrum	Daily, 15 min	Team sync on progress, impediments, and next steps
Sprint Review	End of Sprint	Demo to stakeholders; gather feedback

Downloaded by The Mythiology (themythiology@gmail.com)

Sprint Retrospective	End of Sprint	Inspect and improve process, teamwork, tools

Artifacts

Artifact	Description
Product Backlog	Ordered list of features, bugs, enhancements (maintained by Product Owner)
Sprint Backlog	List of tasks selected for current Sprint (owned by Development Team)
Increment	The working software/features completed in the Sprint

SCRUM METRICS AND MEASUREMENTS

---

1. Story Points

- Unit of **effort estimation**
- Based on **relative complexity, risk, and effort** (not time)
- Usually estimated via **Planning Poker** using the Fibonacci scale:
  1, 2, 3, 5, 8, 13, 21...

---

2. Velocity

Definition:

Average number of story points completed in a Sprint.

Formula:

Velocity = (Total story points completed over X sprints) ÷ X

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

Example:

- Sprint 1: 18 points
  Sprint 2: 22 points
  Sprint 3: 20 points

Velocity = (18 + 22 + 20) ÷ 3 = 20 points per sprint

---

3. Progress Estimation Using Velocity

Goal: Estimate how many Sprints are needed to finish the remaining work.

Formula:

Number of Sprints = Total Remaining Story Points ÷ Velocity

Example:

- Project total = 240 points
- Team velocity = 24 points/sprint

Number of Sprints = 240 ÷ 24 = 10 sprints needed

---

4. Burndown Chart

Definition:
A **graph showing remaining work over time** in a Sprint.

- X-axis: Time (Sprint days)
- Y-axis: Remaining Story Points
- Should **slope down** if work is progressing
- Helps identify bottlenecks or delays

How to Interpret:

Pattern	Meaning

Downloaded by The Mythiology (themythiology@gmail.com)

Straight line down	On track
Flat segments	No progress during those days
Sudden drop	Large task completed suddenly
Burndown above ideal	Behind schedule

ADDITIONAL TECHNICAL CONCEPTS

Sprint Goal

- A **short objective** for the Sprint
  Helps align team on **why** the features are being built
- Evaluated during **Sprint Review**

Definition of Done (DoD)

- A **shared checklist** defining when a task is truly complete
- May include:
  - Code written
  - Code reviewed
  - Tests passed
  - Integrated into main branch
  - Accepted by Product Owner

Exam Application Scenarios

Question Type	**What to Show**
Estimate Sprint count	Use Points ÷ Velocity
Explain velocity	Define as average completed points/sprint

This document is available on studocu
Downloaded by The Mythiology (themythiology@gmail.com)

Interpret burndown chart	Identify progress or bottlenecks
Calculate average velocity	Use (S1 + S2 + ... + Sn) / n
Draw Scrum structure	Show roles, artifacts, events, flow
Compare Scrum vs Waterfall	List flexibility, feedback, delivery cycle
Explain how Scrum handles changing requirements	Show continuous reprioritization of backlog

Summary Table: Scrum Key Elements

Category	**Key Item**	**Notes**
Roles	Product Owner, Scrum Master, Team	Clear responsibility split
Events	Sprint, Planning, Daily, Review, Retro	Time-boxed and structured
Artifacts	Product Backlog, Sprint Backlog, Increment	All tracked and visible
Estimation	Story Points	Relative, not absolute time
Measurement	Velocity, Burndown Chart	Track team productivity
Planning	Sprint Planning + Goal	Select scope per Sprint
Process Control	Retrospective	Continuous improvement

Downloaded by The Mythiology (themythiology@gmail.com)

SC2006 BLACK-BOX TESTING

What is Black-Box Testing?

Black-box testing means **you do not look at the code**. You only look at:

- The **inputs** a function takes
- The **expected outputs** or behavior

You treat the software like a sealed box:
"Put this input in → I should get this output."

You're testing if the system **does what it's supposed to do**, based on its **requirements**, not its internal logic.

---

1. Equivalence Class Testing (ECT)

What does "Equivalence" mean?

An **equivalence class** is a group of input values that the system **treats the same way**. If one value in the group passes or fails, **all others in that group are assumed to behave the same**.

You divide the **entire input space** into **valid** and **invalid** groups.

Why Use It?

Because **you don't need to test every single value**. You just pick **one from each group**, and that's enough.

Example 1: Age Classification System

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

Let's say you're building a program to classify people:

If age < 13 → “Child”
If 13 ≤ age < 18 → “Teen”
If 18 ≤ age ≤ 65 → “Adult”
If age > 65 → “Senior”

Equivalence Classes (EC):

Class Description	Input Range	Valid/Invalid
Negative age	age < 0	Invalid
Child age	0 ≤ age < 13	Valid
Teen age	13 ≤ age < 18	Valid
Adult age	18 ≤ age ≤ 65	Valid
Senior age	age > 65	Valid

Test Cases (pick one value per class):

- Invalid: `-1`
- Valid Child: `5`
- Valid Teen: `15`
- Valid Adult: `30`
- Valid Senior: `70`

---

2. Boundary Value Testing (BVT)

What is a Boundary?

A **boundary** is a number where the system’s behavior **changes**.

In the example above:

Downloaded by The Mythiology (themythiology@gmail.com)

- At `13`, we change from Child → Teen
- At `18`, we change from Teen → Adult
- At `66`, we enter Senior class

Boundaries are where **bugs often happen**, like off-by-one errors.

What is Boundary Value Testing?

You test **values at and near** the boundary.

For age example:

Boundary	Test Values	Why
13	12, 13, 14	Just before, at, and after
18	17, 18, 19	Same logic
65/66	65, 66	Adult to Senior switch

This catches logic errors like:

if (age < 18) // should be <= ?

BVT Rule of Thumb

For each boundary `X`, test:

- `X - 1`
- `X`
  `X + 1`

What are Discrete vs Range Inputs?

This document is available on studocu
Downloaded by The Mythiology (themythiology@gmail.com)

Range Inputs:

- Numeric values with a **range**
- Example: Age 0–100, Salary 3000–10000
- Use **Boundary Value Testing**

Discrete Inputs:

- Picked from a **list**, not numbers
- Example: Room type = {1-room, 2-room, 3-room, 4-room}
- **No boundary values** apply here. You use only **Equivalence Class Testing**

Valid vs Invalid Classes

Valid Class:

- Input that's **allowed**
- The system is **expected to work** for it

Invalid Class:

- Input that's **not allowed**
- The system should **reject it**, show error, or return "Invalid"

Important Rule:

Only 1 invalid input per test case

Why? Because if the test fails, you won't know **which invalid input** caused it. Keep tests isolated.

Downloaded by The Mythiology (themythiology@gmail.com)

Sample Test Case Table

Suppose we have a function:

Input: salary (must be 3000–10000), roomType (1-room to 4-room)
Output: whether loan is approved

Test ID	Salary	Room Type	Expected Result	Reason
TC1	2000	3-room	Invalid Input	Salary too low
TC2	3000	2-room	Approved	Lower boundary salary
TC3	9999	4-room	Approved	Near upper salary
TC4	10001	1-room	Invalid Input	Salary too high
TC5	5000	5-room	Invalid Input	Room type invalid
TC6	7000	3-room	Approved	Normal values

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

SC2006 WHITE BOX TESTING

What is White-Box Testing?

White-box testing is when you **look inside the code** and test it based on:

- The **logic**
- The **decisions**
- The **paths** the program can take

You test whether the **internal structure** of the code is working as expected — not just the input/output.

Why Use It?

- To **check that every line of code is executed**
- To find bugs in **logic, branches, and loops**
- To write test cases based on **actual control flow**

White-Box Testing Techniques

1. Statement Coverage

- Make sure **every line** is run at least once
- Weakest level of white-box testing

---

2. Branch Coverage (Decision Coverage)

- Make sure **every true/false branch** is tested

Example:

```
if (x > 0) { ... }
```

Downloaded by The Mythiology (themythiology@gmail.com)

- You need one test where `x > 0` (true), and one where `x <= 0` (false)

---

3. Condition Coverage

- For **compound conditions** (like `if (a > 0 && b > 0)`), test each condition separately
- Test:
  - `a > 0` true and false
  - `b > 0` true and false

---

4. Path Coverage / Basis Path Testing

- Identify **independent paths** in the code using a **Control Flow Graph (CFG)**
- Use **Cyclomatic Complexity (CC)** to determine **how many test cases** are needed

---

How to Draw a Control Flow Graph (CFG)

Rules:

- **Each node** is a statement or decision
- **Each arrow (edge)** is a possible path
- Start with an **Entry node**
- End with an **Exit node**

Example Code:

```
int max(int a, int b) {
  int result;
  if (a > b)
    result = a;
  else
    result = b;
  return result;
}
```

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

CFG (text form):

[1] int result;
[2] if (a > b)
[3] result = a;
[4] result = b;
[5] return result;

Edges:

- 1 → 2
- 2 → 3 (true)
- 2 → 4 (false)
- 3 → 5
- 4 → 5

How to Calculate Cyclomatic Complexity (CC)

Method 1: Graph formula

CC = E − N + 2

- E = number of edges
- N = number of nodes

Method 2: Count decisions

CC=D+1

- D = number of decision points (e.g., if, while)

For above example:

Downloaded by The Mythiology (themythiology@gmail.com)

- Nodes = 5
- Edges = 5
- CC = 5 − 5 + 2 = **2**

→ You need **2 test cases** to test all independent paths

What is a Basis Path?

- A **unique, independent route** through the CFG
- Each new path should add something **new** (like going through a different branch)

Basis Paths for Example:

Path ID	Path
P1	1 → 2 (true) → 3 → 5
P2	1 → 2 (false) → 4 → 5

Write Test Cases from Paths

Create input values that **force the program to go through each path**.

Example:

if (a > b)

So test:

- `a = 5, b = 3` → P1
- `a = 2, b = 4` → P2

Your test case table:

This document is available on studocu

Downloaded by The Mythiology (themythiology@gmail.com)

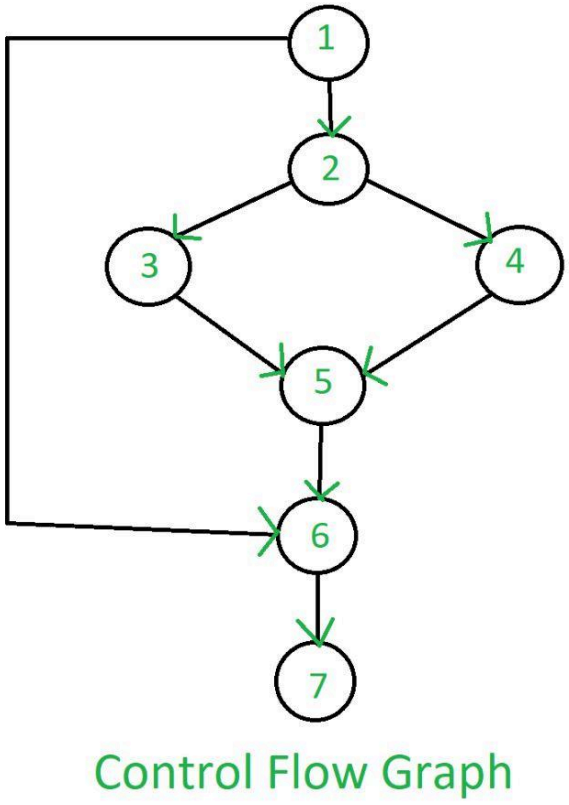
Test ID	Input (a, b)	Expected Output	Path Covered
TC1	(5, 3)	5	P1
TC2	(2, 4)	4	P2

How to Handle Loops in CFG

- Loops (e.g., while, for) introduce **cycles** in the graph
- Only test:
    - 0 iterations
    - 1 iteration
    - (Optional: many iterations if needed)

Because full path coverage in loops = **infinite paths → not feasible**

Another visual example of CFG



Downloaded by The Mythiology (themythiology@gmail.com)